Kubernetes Networking & Ingress on Minikube — Hands-On Guide

Oct 18, 2026 · @Yetty

Session overview

By the end of this session every participant will open the Cloud Native Notes app at `http://notes.local/` and reach its API at `http://notes.local/api/notes` through a Kubernetes Ingress, with no `kubectl port-forward`. The session runs about 3 hours on a MacBook using Minikube.

What participants will be able to do

- Explain how a request travels from a browser to a pod: Ingress controller, Ingress rule, Service, Pod.
- Tell apart the three Service types: ClusterIP, NodePort and LoadBalancer.
- Install the NGINX Ingress controller and route traffic by host name and path.
- Prove that the frontend reaches the backend through Kubernetes DNS.
- Explain why a LoadBalancer Service stays `<pending>` on a laptop.

Agenda

Block	Time	What happens
Concepts	30 min	Networking ideas, traffic diagram
Setup	30 min	Install tools, start Minikube (ideally done before the session)
Labs 1–2	40 min	Build the image, deploy the app
Labs 3–4	45 min	Ingress controller, reach the app from the Mac
Lab 5 + wrap-up	25 min	Service-to-service traffic, LoadBalancer question, Q&A
Extra challenges	Take-home	Gateway API, TLS

> Tip for facilitators: send the *Prerequisites and installation* section a few days early. Docker Desktop downloads are large and slow on shared Wi-Fi.

Minikube or Kind on a Mac?

Use Minikube for this beginner session. On macOS, Ingress works with two commands (`minikube addons enable ingress` and `minikube tunnel`), while Kind needs the cluster rebuilt with special port mappings first.

	Minikube	Kind
Install	`brew install minikube`	`brew install kind`
Ingress controller	Built-in addon, one command	Apply a manifest yourself
Reaching Ingress from the Mac	`minikube tunnel`, then `127.0.0.1`	Must recreate the cluster with `extraPortMappings` (the README's "Herculean task")
LoadBalancer Services	Work through `minikube tunnel`	Stay `<pending>` unless you add an extra tool
Loading a local image	`minikube image build`	`kind load docker-image`
Extras for teaching	Dashboard, addons, `minikube service --url`	Lighter, fast multi-node clusters
Best for	Beginners, single-node labs	CI pipelines, multi-node experiments

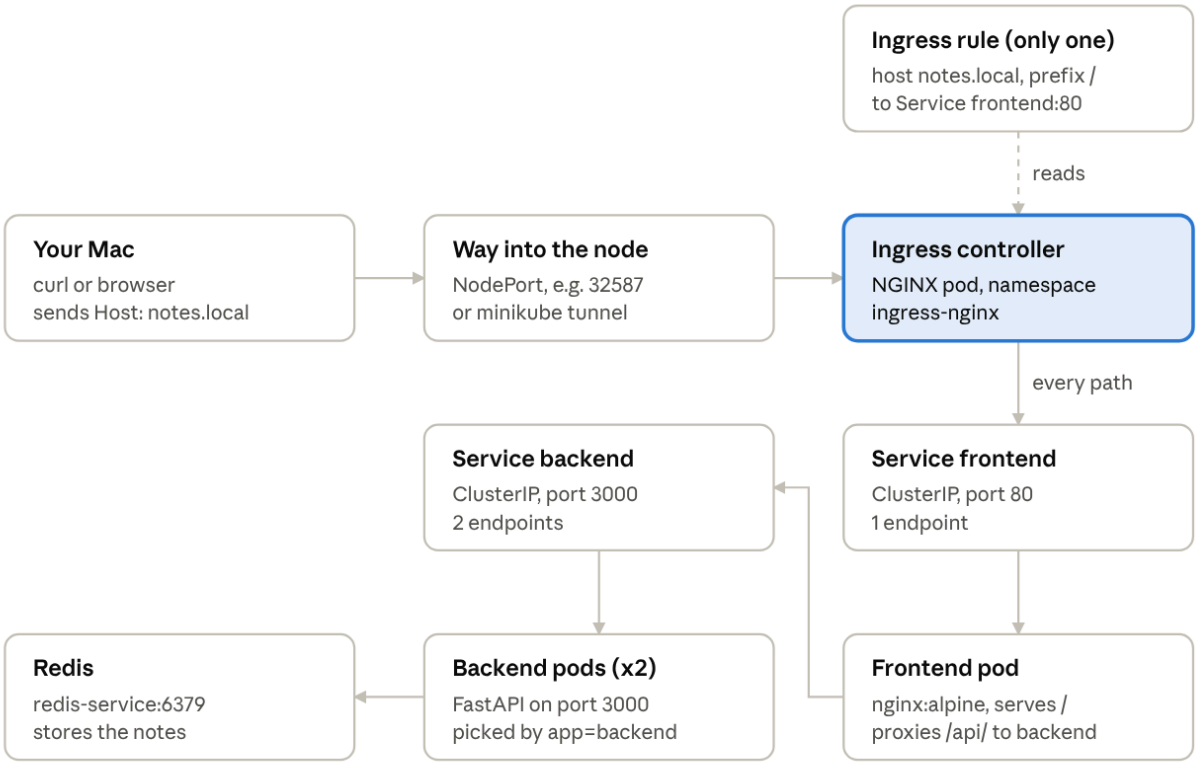
Why the Mac matters. Both tools run the Kubernetes node inside a Docker container. On macOS, Docker itself runs inside a hidden Linux VM, so the node IP (for example `192.168.49.2`) is not reachable from your Mac's terminal or browser. Minikube ships helpers that bridge this gap; Kind leaves it to you.

Note about the project README. The README was written with Kind in mind: it talks about worker nodes and curling `<WORKER-NODE-IP>:<NODEPORT>`. This guide gives the Minikube equivalent of every step, and points out where the commands differ. Participants who prefer Kind follow the Kind track section after Lab 5.

Concepts to understand before you begin

Every request in this lab passes through four layers: Ingress controller, Ingress rule, Service, Pod. Learn these terms first and every command later will make sense.

One front door: Ingress to frontend, frontend to backend



request path · Mac to backend pod

Only the Ingress controller is reachable from outside; both Services stay ClusterIP and are reached through the controller or through DNS.

Pod. The smallest thing Kubernetes runs, one or more containers sharing an IP address. Pod IPs change every time a pod restarts, so nothing should talk to a pod IP directly.

Service. A stable name and IP in front of a group of pods. It picks its pods with a *label selector* and spreads traffic across them. There are three types you will meet today:

Service type	Reachable from	Typical use	In this lab
ClusterIP (default)	Inside the cluster only	Pod-to-pod traffic	`backend` on port 3000
NodePort	Every node's IP on a port 30000–32767	Quick external access, labs	Ingress controller on Minikube
LoadBalancer	An external IP from a cloud load balancer	Production on AWS, GCP, Azure	Shows `<pending>` until `minikube tunnel` runs

Each type builds on the one above it: a NodePort Service also has a ClusterIP, and a LoadBalancer Service also has a NodePort.

Two more kinds exist that you will not use today: **ExternalName** (a DNS alias for something outside the cluster, such as `my-db` → `db.example.com`) and **headless** Services (`clusterIP: None`, where DNS returns every pod's IP; used with StatefulSets like databases). Ingress is *not* a Service type: it sits in front of Services and routes HTTP by host and path.

Endpoints. The list of pod IPs currently behind a Service. The Service IP is virtual: no pod owns it. Kubernetes fills the endpoint list from the Service's label selector, keeping only *ready* pods, and kube-proxy forwards each connection to one of them. Two backend replicas means two endpoints; scale to three and a third IP appears automatically. **When a Service "doesn't work", check its endpoints first:** an empty list means a selector/label mismatch or no ready pods.

```
Service backend (selector app=backend, 10.109.73.219:3000)
   └─ Endpoints: 10.244.0.152:3000, 10.244.0.153:3000   ← the two backend
pods
```

Kubernetes DNS (CoreDNS). Every Service gets a DNS name. Inside the same namespace, the short name works: the frontend calls `http://backend:3000`. The full name is `backend.cloud-native-notes.svc.cluster.local`.

Ingress (the rule). A Kubernetes object that maps a host name and path to a Service. This project has exactly one rule: host `notes.local`, path `/` (prefix, so it matches everything) → Service `frontend` on port 80. The Ingress never talks to the backend. The frontend's own NGINX (from `frontend-nginx-configmap.yaml`) forwards `/api/...` to `backend:3000`. On its own, an Ingress does nothing; it is only a set of rules.

Ingress controller (the engine). A pod, here running NGINX, that watches Ingress objects and turns them into a live reverse-proxy configuration. Without a controller installed, Ingress rules are ignored. This is why step one of the README is "Install Ingress Controller".

The Host header. One controller can serve many websites on one IP. It decides which rule applies by reading the `Host` header of each request. That is why the README curls with `-H "Host: notes.local"`: it tells NGINX which site you want without needing DNS.

Gateway API. The newer successor to Ingress. It splits responsibilities into `GatewayClass`, `Gateway` and `HTTPRoute` objects and supports features Ingress needs vendor annotations for, such as traffic splitting and header matching. It is covered in the extra challenges.

Networking glossary — every term in one place

A quick reference to keep open during the labs. Each term says what it is and where you meet it in this project.

Addresses and ports

Term	What it is	Where you use it
IP address	A number that identifies a machine or pod on a network	Every pod, node and Service has one
Port	A numbered door on an IP; one IP can run many programs on different ports	Backend 3000, frontend 80, Redis 6379
`localhost` / `127.0.0.1`	"This same machine": on the Mac it means the Mac, inside `minikube ssh` it means the node	Why `localhost:32587` works on the node but is refused on the Mac
Pod IP	The pod's own address, for example `10.244.0.152`; changes every time the pod is recreated	Shown in endpoints and `kubectl get pods -o wide`; never hard-code it
Node IP	The address of the machine running the pods, for example `192.168.49.2` (`minikube ip`)	Shown as the Ingress `ADDRESS`; not reachable from a Mac with the Docker driver
Namespace	A named group of objects; Service names are short-names inside it	`cloud-native-notes`, `ingress-nginx`

Services and the four port fields

Term	What it is	Where you use it
Service	A stable name and virtual IP in front of a set of pods	`backend`, `frontend`, `redis-service`
Labels and selector	Labels are tags on pods (`app: backend`); a selector is how a Service or Deployment finds pods with that tag	Must match, or the Service has no endpoints
ClusterIP	Default Service type; reachable only inside the cluster	All three app Services

Term	What it is	Where you use it
NodePort	Opens the same port (30000–32767) on every node	The Ingress controller's Service: 32587 (HTTP), 30282 (HTTPS)
LoadBalancer	Asks a cloud for an external IP; stays `<pending>` on a laptop	The `hello` demo; `minikube tunnel` fills it in
ExternalName	A DNS alias to something outside the cluster	Not used here
Headless Service	`clusterIP: None`; DNS returns each pod's IP	Databases with StatefulSets; not used here
`port`	The port callers use on the Service	Ingress `port.number: 80`; `backend:3000`
`targetPort`	The pod port the Service forwards to	Must equal `containerPort`
`containerPort`	The port the app listens on inside the container	`3000` in `backend-deployment.yaml`
`nodePort`	The port opened on each node for a NodePort Service	Random unless you set it
Endpoints / EndpointSlice	The live list of ready pod IP:ports behind a Service	`kubectl get endpoints`; check first when a Service fails
Readiness probe	A health check; only ready pods become endpoints	A crashing pod drops out of the list
kube-proxy	Runs on every node and turns a Service IP into a real pod IP	Why a virtual ClusterIP works at all
Replicas	How many copies of a pod run	2 backend replicas = 2 endpoints

Names and DNS

Term	What it is	Where you use it
DNS	Turns names into IP addresses	`backend` → `10.109.73.219`
CoreDNS	The DNS server inside Kubernetes	Answers the `nslookup backend` in Lab 5
FQDN (full name)	A Service's complete DNS name	`backend.cloud-native-notes.svc.cluster.local`

Term	What it is	Where you use it
`/etc/hosts`	A file on the Mac that maps names to IPs before DNS is asked	`127.0.0.1 notes.local`
nip.io	A public DNS service where the IP is written in the name	`notes.127-0-0-1.nip.io` → `127.0.0.1` (week 6 script)

HTTP and routing

Term	What it is	Where you use it
HTTP / HTTPS	Web protocol; HTTPS is the encrypted version (port 443)	Plain HTTP to 443 gives *400 Bad Request*
TLS certificate	Proves a site's identity and enables HTTPS	The controller's default one is self-signed, so use `curl -k`
Host header	The site name a request asks for; how one IP serves many sites	`-H "Host: notes.local"`; a browser sends the address-bar name
Path and `pathType: Prefix`	The URL part after the host; Prefix `/` matches every path	The single Ingress rule
Reverse proxy	A server that receives requests and passes them to other servers	The Ingress controller and the frontend NGINX are both proxies
`proxy_pass`	The NGINX setting that forwards a path elsewhere	`location /api/` → `http://backend:3000` in the frontend config
Status codes	The server's answer: 200 OK, 301 moved, 400 bad request, 404 not found, 503 no ready pods	See *Reading the error codes* in Lab 4

Ingress and getting traffic in

Term	What it is	Where you use it
Ingress	Rules mapping host + path to a Service	`ingress.yaml`

Term	What it is	Where you use it
IngressClass	Names which controller handles an Ingress	`ingressClassName: nginx`
Ingress controller	The pod that reads Ingress rules and becomes the proxy	`ingress-nginx-controller`
Admission webhook	The controller's check that rejects broken Ingress objects before they're saved	The `-admission` Service; the *failed calling webhook* error
Gateway API	The successor to Ingress (`GatewayClass` , `Gateway` , `HTTPRoute`)	Extra challenge 1
`kubectl port-forward`	A temporary private tunnel from your machine to one pod or Service	What Ingress lets you stop using
`minikube service --url`	A temporary tunnel from a random Mac port to a NodePort	Lab 4 Method B: first URL = HTTP
`minikube tunnel`	Opens ports 80/443 on the Mac and gives LoadBalancers an IP	Lab 4 Method C, week 6 script
`extraPortMappings`	Kind's way to publish node ports on the Mac at cluster creation	Kind track K1
CNI	The plug-in that gives pods their IPs and connects them	Built into Minikube and Kind; nothing to set up
NetworkPolicy	Firewall rules saying which pods may talk to which	Not used here; by default every pod can reach every pod

Prerequisites and installation (macOS)

You need four tools: Homebrew, Docker Desktop, kubectl and Minikube. Plan for at least 8 GB of RAM on the Mac and 20 GB of free disk.

1. **Check your chip.** Apple menu → *About This Mac*. "Apple M1/M2/M3/M4" means Apple Silicon; "Intel" means Intel. Homebrew picks the right build automatically, but Docker Desktop has two downloads.
2. **Install Homebrew** (skip if `brew --version` already works):

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

On Apple Silicon, run the two `echo ... >> ~/.zprofile` lines the installer prints at the end, then open a new terminal.

3. **Install Docker Desktop.** Download it from docker.com (pick Apple chip or Intel chip), drag it to Applications and open it once. Wait for the whale icon in the menu bar to say *Docker Desktop is running*. In *Settings → Resources*, give it at least 4 CPUs and 6 GB of memory.
4. **Install kubectl and Minikube:**

```
brew install kubectl minikube
```

5. **Verify the tools:**

```
docker version          # shows both Client and Server
kubectl version --client
minikube version
```

6. **Optional helpers** that beginners find useful:

```
brew install k9s jq      # terminal UI for clusters, JSON pretty-printer
```

> If `docker version` shows only the Client and an error for Server, Docker Desktop is not running. Open it from Applications and wait for it to finish starting.

Project layout

The repository has three parts: the application code in `backend/`, the Kubernetes manifests in `k8s/`, and the `README.md` with the week's tasks.

```
week5/  (or week6/)
├── README.md                      # the week's tasks and verify steps
├── verify-week6.sh                # week 6: automated checks
├── backend/                       # FastAPI app + Dockerfile (port 3000)
│   └── app/  main.py config.py database.py metrics.py
└── k8s/
    ├── namespace.yaml
    ├── configmap.yaml  secret.yaml            # backend settings, Redis
password
    ├── backend-deployment.yaml  backend-service.yaml
    ├── frontend-deployment.yaml               # Deployment AND its Service
(separated by ---)
    ├── frontend-configmap.yaml                # the HTML page
    ├── frontend-nginx-configmap.yaml          # NGINX: serves /, proxies
/api/ to backend:3000
    ├── redis-deployment.yaml                  # Deployment AND redis-service
    └── ingress.yaml                           # one rule: notes.local / →
frontend:80
```

One YAML file can hold several objects separated by `---`. That is why there is a `frontend` Service but no `frontend-service.yaml`.

Before the labs, take two minutes to read the manifests. These commands show the details you will need later:

```
ls k8s/
grep -n "kind:" k8s/*.yaml            # which objects get created
grep -n "image:" k8s/*.yaml           # image names you must build
grep -n -A3 "ingressClassName\|host:" k8s/*.yaml   # Ingress class and host
```

From the README we already know these values:

Item	Value
Namespace	`cloud-native-notes`
Backend Service	`backend`, port 3000
Ingress host	`notes.local`
API path	`/api/` (with the slash), forwarded by the frontend NGINX; the frontend page is served at `/`
Expected API reply	`{"message":"Cloud Native Notes","endpoints":[...]}`

Check one thing in the Ingress file: `ingressClassName: nginx`. The Minikube addon registers the class `nginx`, so the value must match or the rule is ignored.

Lab 1 — Start the cluster and build the backend image

Goal: a running one-node Minikube cluster that already holds the backend image, so no registry is needed.

1. **Start Minikube** with the Docker driver:

   ```
   minikube start --driver=docker --cpus=2 --memory=4096
   ```

   The first start downloads about 1 GB. Memory must be lower than what Docker Desktop has been given.
2. **Confirm the cluster is up:**

   ```
   kubectl get nodes -o wide
   minikube status
   ```

   Expect one node named `minikube` with status `Ready`.
3. **Build the backend image inside Minikube.** Use exactly the image name and tag you found with `grep image:` in the project layout step. This project's backend Deployment uses `cloud-native-notes:v5`:

   ```
   minikube image build -t cloud-native-notes:v5 ./backend
   minikube image ls | grep cloud-native-notes
   ```

   Run this from the week folder (`week5` or `week6`), never from inside `backend/`. Your prompt tells you where you are: `yomex@... week5 %`. A build started from the wrong folder prints nothing and builds nothing, and the pods keep running the old image. Always build the exact tag your manifest names; week 6's `backend-deployment.yaml` uses `cloud-native-notes:3.0`.
   Alternative: run `eval $(minikube docker-env)` and then a normal `docker build`. That points your terminal's Docker at the daemon inside Minikube.
4. **Check the pull policy.** If the manifest uses the `:latest` tag, Kubernetes tries to download it from Docker Hub and fails. Use a fixed tag such as `v1`, or set `imagePullPolicy: IfNotPresent` in the Deployment.

Checkpoint: `kubectl get nodes` shows `Ready` and `minikube image ls` lists your backend image.

Lab 2 — Deploy the application

Goal: frontend and backend pods running, with Services and an Ingress created. The Ingress will not work yet, on purpose.

1. **Apply every manifest in the folder:**

   ```
   kubectl apply -f k8s/
   ```

   If you see `namespaces "cloud-native-notes" not found`, run the same command again: the namespace was created during the first run.
2. **Watch the pods start** (press Ctrl+C when all are `Running`):

   ```
   kubectl get pods -n cloud-native-notes -w
   ```
3. **Look at the Services:**

   ```
   kubectl get svc -n cloud-native-notes
   ```

   Note the `TYPE` column. The backend should be `ClusterIP`: it has no way in from outside the cluster.
4. **Check that each Service found its pods:**

   ```
   kubectl get endpoints -n cloud-native-notes
   ```

   Each Service should list one or more `IP:port` pairs. An empty list means the Service selector does not match the pod labels.
5. **Look at the Ingress:**

   ```
   kubectl get ingress -n cloud-native-notes
   ```

   The `ADDRESS` column is empty. Discussion question: *why?* (Answer: no Ingress controller is installed yet, so nothing is reading this rule.)

Checkpoint: all pods `Running`, endpoints filled in, Ingress present with no address.

Lab 3 — Install the Ingress controller

Goal: an NGINX Ingress controller running in the `ingress-nginx` namespace and picking up the app's Ingress rule.

1. **Enable the Minikube addon.** This replaces the README's `kubectl apply -f .../provider/cloud/deploy.yaml` step:

   ```
   minikube addons enable ingress
   ```

Use the addon *or* the README manifest, never both: two controllers fight over the same Ingress objects.

2. **Wait until the controller is ready** (same command as the README):

```
kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=120s
```

Expected: `pod/ingress-nginx-controller-xxxxx condition met`.

3. **See what was installed:**

```
kubectl get pods,svc -n ingress-nginx
kubectl get ingressclass
```

You will see two Services, because the controller pod has two jobs:

Service	Type	Used by	Job
`ingress-nginx-controller`	NodePort, for example `80:32587/TCP,443:30282/TCP`	Browsers and curl	Carries website traffic: HTTP on 80, HTTPS on 443
`ingress-nginx-controller-admission`	ClusterIP, `443/TCP`	The Kubernetes API server	Validates every Ingress before it is saved (the webhook)

The NodePort numbers are not in any of your files. The addon's Service says `type: NodePort` without a number, so Kubernetes picks free ports at random from 30000–32767. Every participant gets different numbers, so each person looks up their own HTTP port:

```
kubectl get svc ingress-nginx-controller -n ingress-nginx \
  -o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}{"\n"}'
```

The two `admission-create` and `admission-patch` pods showing `Completed` are one-off setup jobs. That is their normal final state.

4. **Check the Ingress again:**

```
kubectl get ingress -n cloud-native-notes
kubectl describe ingress -n cloud-native-notes
```

`ADDRESS` now shows an IP (it can take up to a minute). `describe` lists each rule: host, path and the backend Service it points at.

5. **Peek at the generated NGINX config** (optional, but it makes the idea concrete):

```
# run this whole line on your Mac; kubectl exec runs grep inside the
controller pod
kubectl exec -n ingress-nginx deploy/ingress-nginx-controller -- grep -n
"server_name notes.local" /etc/nginx/nginx.conf
```

Expected: a line like `server_name notes.local ;`. If you see `No such file or directory`, you ran only the `grep` part on your Mac.

Checkpoint: controller pod `Running`, Ingress shows an address.

Lab 4 — Reach the app through Ingress

Goal: get `HTTP/1.1 200 OK` and the API's JSON through the Ingress controller. Do the three methods in order: each one goes one step closer to a real browser.

On a Mac, `curl http://$(minikube ip):<NODEPORT>` from your terminal hangs, because the node lives inside Docker's hidden VM. This is the single most common beginner trap.

Method A — NodePort from inside the node (closest to the README)

```
minikube ssh                      # prompt changes to docker@minikube:~$
curl -v -H "Host: notes.local" http://localhost:<NODEPORT>/
curl -v -H "Host: notes.local" http://localhost:<NODEPORT>/api/notes
exit                              # back to your Mac
```

Replace `<NODEPORT>` with your HTTP NodePort from Lab 3, such as `32587`. Here you are standing on the node, exactly like the README's worker-node curl. The same command run on your Mac (prompt `yomex@...`) fails with *Connection refused*: the port only exists on the node.

Use `/api/` or `/api/notes`, not `/api`. The frontend NGINX has `location /api/` with a trailing slash, so `/api` answers `301 Moved Permanently` to `/api/`. The footer tells you who answered: `nginx/1.31.1` (with a version) is the frontend pod; plain `nginx` is the Ingress controller.

Method B — A temporary tunnel to the controller

```
minikube service ingress-nginx-controller -n ingress-nginx --url
```

It prints two URLs, one per controller port, each on a random local port, for example `http://127.0.0.1:64516` and `http://127.0.0.1:64517`. **The first is HTTP (port 80); the second is HTTPS (port 443)** and answers *400 The plain HTTP request was sent to HTTPS port*. The note *Because you are using a Docker driver on darwin, the terminal needs to be open* means the tunnel lives only while that command runs. Leave it open and, in a second terminal:

```
curl -v -H "Host: notes.local" http://127.0.0.1:<FIRST-PORT>/
curl -v -H "Host: notes.local" http://127.0.0.1:<FIRST-PORT>/api/notes
```

Now try it without the Host header, or open the same URL in a browser: you get *404 Not Found* from the controller. A browser always sends the name in its address bar (`Host: 127.0.0.1:64516`), which matches no rule. To use a browser with this tunnel, add `127.0.0.1 notes.local` to `/etc/hosts` and open `http://notes.local:<FIRST-PORT>/`. NGINX ignores the port when matching host names.

Method C — `notes.local` in your browser (the README's "Herculean task")

1. Map the name to your Mac:

   ```
   echo "127.0.0.1 notes.local" | sudo tee -a /etc/hosts
   ```

2. Start the tunnel in its own terminal and leave it running (it asks for your Mac password):

   ```
   minikube tunnel
   ```

3. Test it:

   ```
   curl -v http://notes.local/
   curl -v http://notes.local/api/notes
   ```

   Then open `http://notes.local/` in the browser. Type the full address with `http://`, or Chrome may switch to HTTPS or run a search. The page shows *Cloud Native Notes — Week 5: Ingress* and loads its notes from `/api/notes` through the frontend's proxy.

Expected results. `/` returns `200 OK` with the HTML page. `/api/notes` returns `200 OK` with a JSON list of notes. `/api/` returns the README's JSON:

```
< HTTP/1.1 200 OK
< Content-Type: application/json
{"message":"Cloud Native Notes","endpoints":
["/health","/api/notes","/api/notes/{id}"]}
```

Notice what you did *not* do: no `kubectl port-forward`. Traffic entered through one front door and NGINX routed it.

Checkpoint: 200 OK and the JSON from at least Method A or B; Method C if time allows.

Reading the error codes. Each response points to exactly one misunderstanding, which makes this table worth showing in the session:

You see	Who answered	Meaning	Fix
`Connection refused` on `localhost:<NODEPORT>`	Nobody	Command ran on the Mac; the NodePort only exists on the node	`minikube ssh` first, or use Method B/C
`400 Bad Request` — *plain HTTP request was sent to HTTPS port*	Ingress controller	`http://` sent to the 443 port (30282, or the second `--url` URL)	Use the HTTP port / first URL, or `curl -k https://...`
`404 Not Found`	Ingress controller	Host was not `notes.local` (missing `-H`, or a browser on an IP address)	Add `-H "Host: notes.local"`, or use `/etc/hosts` and the name
`301 Moved Permanently` → `/api/`	Frontend NGINX (`nginx/1.31.1`)	Asked for `/api` without the slash	Use `/api/` or `/api/notes`, or `curl -L`
`200 OK`	Frontend, or backend through the proxy	Everything along the path works	—

Lab 5 — Frontend to backend inside the cluster

Goal: prove that pods talk to each other through Service names and Kubernetes DNS, with no Ingress involved.

1. **Find the frontend pod name:**

   ```
   kubectl get pods -n cloud-native-notes -l app=frontend
   ```

   If the label differs, run `kubectl get pods -n cloud-native-notes --show-labels` and use what you see.
2. **Call the backend from the frontend pod** (the README command):

```
kubectl exec -n cloud-native-notes <frontend-pod-name> -- \
  curl -i http://backend:3000/api/notes
```

Expect `HTTP/1.1 200 OK` and a JSON list of notes.

3. **If the frontend image has no `curl`**, start a throwaway pod instead:

```
kubectl run tmp-curl -n cloud-native-notes --rm -it --restart=Never \
  --image=curlimages/curl -- curl -i http://backend:3000/api/notes
```

4. **See the DNS lookup behind it:**

```
kubectl run tmp-dns -n cloud-native-notes --rm -it --restart=Never \
  --image=busybox:1.36 -- nslookup backend
```

   The answer shows `backend.cloud-native-notes.svc.cluster.local` and the Service's ClusterIP. Compare it with `kubectl get svc backend -n cloud-native-notes`.

5. **Discussion:** the backend is reachable here but not from your Mac. Why? (It is a ClusterIP Service; only the Ingress controller is exposed.)

Checkpoint: 200 OK from inside the cluster and the DNS name resolves to the Service IP.

Kind track — the same labs on a Kind cluster

Kind users follow the same concepts and Labs 2 and 5 unchanged. Only cluster creation, image loading, the Ingress controller and the access step differ. The key rule on a Mac: create the cluster with port mappings for 80 and 443, or `notes.local` will never reach it from your browser.

Install Kind (Docker Desktop and kubectl as in the installation section):

```
brew install kind
kind version
```

K1 — Create the cluster with Ingress port mappings

Save this as `kind-config.yaml` in the project root. It makes one control-plane node ready for Ingress and two workers, matching the README's "worker node" wording:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: notes
nodes:
  - role: control-plane
    kubeadmConfigPatches:
```

```
      - |
        kind: InitConfiguration
        nodeRegistration:
          kubeletExtraArgs:
            node-labels: "ingress-ready=true"
    extraPortMappings:
      - containerPort: 80
        hostPort: 80
        protocol: TCP
      - containerPort: 443
        hostPort: 443
        protocol: TCP
  - role: worker
  - role: worker
```

```
kind create cluster --config kind-config.yaml
kubectl get nodes -o wide        # notes-control-plane + 2 workers, all Ready
```

`extraPortMappings` publishes ports 80 and 443 of the control-plane container on your Mac, the same way `docker run -p 80:80` would. Port mappings cannot be added later: an existing cluster must be deleted and recreated (`kind delete cluster --name notes`).

K2 — Build and load the backend image

Kind nodes cannot see images on your Mac's Docker until you load them:

```
docker build -t cloud-native-notes:v5 ./backend
kind load docker-image cloud-native-notes:v5 --name notes
```

Use the image name from `k8s/` and avoid the `:latest` tag, exactly as in Lab 1. Then deploy the app with Lab 2 unchanged.

K3 — Install the Ingress controller (Kind provider)

Use the **kind** manifest, not the README's **cloud** one. The cloud manifest asks for a LoadBalancer, which stays `<pending>` on Kind. The kind manifest pins the controller to the `ingress-ready` node and binds it to that node's ports 80 and 443:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-
nginx/controller-v1.8.1/deploy/static/provider/kind/deploy.yaml

kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=120s

kubectl get pods -n ingress-nginx -o wide   # NODE = notes-control-plane
```

If `kubectl apply -f k8s/` ran before the controller was ready and failed on the webhook, apply it again now.

K4 — Reach the app

Method A: browser and curl from the Mac (the "Herculean task", solved by K1).

```
echo "127.0.0.1 notes.local" | sudo tee -a /etc/hosts
curl -v http://notes.local/api
```

No tunnel is needed: port 80 on your Mac already leads into the cluster. Open `http://notes.local/` in the browser too.

Method B: the README's NodePort test, done from inside the Docker network. Node IPs such as `172.18.0.3` are not reachable from the Mac, so run curl from a node container:

```
kubectl get svc -n ingress-nginx ingress-nginx-controller   # note the
NodePort for 80
kubectl get nodes -o wide                                   # note a node
INTERNAL-IP
docker exec -it notes-worker \
  curl -v -H "Host: notes.local" http://<NODE-IP>:<NODEPORT>/api
```

Method C: no port mappings and no rebuild allowed. Forward a local port to the controller as a last resort:

```
kubectl port-forward -n ingress-nginx svc/ingress-nginx-controller 8080:80
curl -v -H "Host: notes.local" http://127.0.0.1:8080/api
```

Then run Lab 5 unchanged.

Minikube and Kind command map

Step	Minikube	Kind
Create cluster	`minikube start --driver=docker`	`kind create cluster --config kind-config.yaml`
Load local image	`minikube image build -t <img> ./backend`	`docker build` + `kind load docker-image <img> --name notes`
Ingress controller	`minikube addons enable ingress`	`kubectl apply -f .../provider/kind/deploy.yaml`
Browser access	`minikube tunnel` + `/etc/hosts`	`/etc/hosts` only (ports mapped at creation)
Shell on a node	`minikube ssh`	`docker exec -it notes-worker bash`
LoadBalancer IP	`minikube tunnel`	`cloud-provider-kind` or MetalLB
Delete cluster	`minikube delete`	`kind delete cluster --name notes`

Kind-specific troubleshooting: if `kind create cluster` fails with *port is already allocated*, another program (often a local web server or a Minikube tunnel) holds port 80 on the Mac. Stop it, or map `hostPort: 8080` and browse to `http://notes.local:8080`.

Week 6 — Pass the verification script on Minikube

Week 6 ships `verify-week6.sh`, which runs about 33 checks: the controller, the frontend ConfigMaps, the Ingress rule, backend and Redis health, then real traffic and a create/read/update/delete test through the Ingress. Run it from the `week6` folder:

```
chmod u+x verify-week6.sh
./verify-week6.sh
```

The script was written for the **Kind** setup (see the Kind track). On a fresh Minikube cluster it typically scores about 78%: everything inside the cluster passes, and the failures come from three Kind assumptions.

Script expects (Kind)	Minikube by default	Checks that fail	Fix
A node labelled `ingress-ready=true`	No such label	4 (second part)	Fix 1 below

Script expects (Kind)	Minikube by default	Checks that fail	Fix
The controller's node **name** contains `control-plane` (Kind names it `<cluster>-control-plane`)	The node is named `minikube`	4 and 11	*Getting 38/38* below
The app answering on the Mac at `127.0.0.1:80`, host `notes.127-0-0-1.nip.io`	Nothing on port 80 until `minikube tunnel`; the Ingress only knows `notes.local`	8, 9 and 10	Fixes 3 and 4 below

Checks 9 (API) and 10 (CRUD) are not separate bugs: once the script cannot reach the Ingress in check 8, every later traffic test fails too.

About nip.io. `notes.127-0-0-1.nip.io` is a public DNS name that always resolves to `127.0.0.1`, so no `/etc/hosts` edit is needed. The request still carries `Host: notes.127-0-0-1.nip.io`, so the Ingress needs a rule for that name.

The four fixes

1. **Label the Minikube node:**

```
kubectl label node minikube ingress-ready=true
```

2. **Pin the controller to that node** (the patch the script suggests). Minikube has one node, so the pod does not move; this only satisfies the check:

```
kubectl patch deployment -n ingress-nginx ingress-nginx-controller \
  --type=merge \
  -p '{"spec":{"template":{"spec":{"nodeSelector":{"ingress-
ready":"true","kubernetes.io/os":"linux"}}}}}'
kubectl rollout status deployment/ingress-nginx-controller -n ingress-
nginx
```

3. **Add the nip.io host to `k8s/ingress.yaml`**, keeping `notes.local`:

```
# replace the WHOLE file with this; keep the first four lines
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: cloud-native-notes-ingress
  namespace: cloud-native-notes
spec:
  ingressClassName: nginx
  rules:
```

```
  - host: notes.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend
            port:
              number: 80
  - host: notes.127-0-0-1.nip.io
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend
            port:
              number: 80
```

If `kubectl apply` says *apiVersion not set, kind not set*, the header lines are missing: every manifest needs `apiVersion`, `kind` and `metadata` before `spec`.

```
kubectl apply -f k8s/ingress.yaml
```

4. **Open port 80 on the Mac** in a separate terminal, and leave it running (it asks for your password):

```
minikube tunnel
```

Check, then rerun

```
curl -s http://notes.127-0-0-1.nip.io/ | grep "Cloud Native Notes"
curl -s http://notes.127-0-0-1.nip.io/api/notes
./verify-week6.sh
```

With the tunnel running, checks 8 to 10 pass and the score reaches 36/38 (94%). If check 8 fails with the tunnel open, see what holds port 80 with `sudo lsof -i :80`.

Why checks 4 and 11 still fail: they test the node's name

Reading `verify-week6.sh` shows that both checks only look at the name of the node running the controller:

```
NODE=$(kubectl get pods -n ingress-nginx -l
app.kubernetes.io/component=controller \
       -o jsonpath='{.items[0].spec.nodeName}')
if echo "$NODE" | grep -q "control-plane"; then ...   # passes only if the
NAME contains "control-plane"
```

The label and the nodeSelector patch cannot change a node's name, so a cluster called `minikube` stops at 36/38 however well it works. Nothing about routing is wrong at that point.

Getting 38/38 on Minikube: name the node `notes-control-plane`

A Minikube node takes its name from the **profile** (`-p`). A new cluster created as `notes-control-plane` therefore passes both checks. This rebuilds the cluster from scratch, so allow about 10 minutes. Run everything from the `week6` folder:

```
# 1. free port 80 and memory: stop the old tunnel (Ctrl+C in its terminal),
then the old cluster
minikube stop

# 2. new cluster whose node is named notes-control-plane
minikube start -p notes-control-plane --driver=docker --cpus=2 --memory=4096
kubectl get nodes                       # NAME: notes-control-plane

# 3. Ingress controller and node label
minikube -p notes-control-plane addons enable ingress
kubectl wait -n ingress-nginx --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller --timeout=180s
kubectl label node notes-control-plane ingress-ready=true

# 4. backend image, then every manifest including subfolders (-R)
minikube -p notes-control-plane image build -t cloud-native-notes:v8
./backend
kubectl apply -R -f k8s/
kubectl apply -R -f k8s/                # second run catches objects that
needed the namespace first
kubectl rollout status deployment/redis -n cloud-native-notes --timeout=180s
kubectl rollout status deployment/backend -n cloud-native-notes --
timeout=180s
```

In a separate terminal, start the tunnel for the new cluster and leave it running:

```
minikube -p notes-control-plane tunnel
```

Then run `./verify-week6.sh`. Expect **38/38: WEEK 6 VERIFICATION PASSED**.

- From now on every `minikube` command needs `-p notes-control-plane`. `kubectl` switches to the new cluster automatically.
- The image tag you build must match `image:` in `k8s/backend/backend-deployment.yaml` (here `v8`).
- Delete the old cluster once you are happy: `minikube delete -p minikube`.
- For a future cohort, start Lab 1 with `minikube start -p notes-control-plane ...` and this step is never needed.

Week 6 debugging case study

Getting from 78% to 38/38 took five layers of fixes. Each one only became visible after the one before was solved, which makes this a strong closing exercise.

#	Symptom	Cause	How it was found	Fix
1	With 2 backend replicas, CRUD fails at random: GET and DELETE 404 right after POST	Each pod kept notes in its own memory; the Service alternated requests between the 2 endpoints	Scaling to 1 replica made CRUD pass	Make every pod use Redis (layers 2–5)
2	Backend env has `REDIS_SERVICE_*` but no `REDIS_HOST`	The running Deployment had no `envFrom`; the app defaulted to `localhost` and fell back to memory	`kubectl exec deploy/backend -- env | grep REDIS`	Apply the Deployment with `envFrom` for `app-config` and `redis-secret`
3	Rollout stuck, `ImagePullBackOff`, `exceeded its progress deadline`	The manifest asked for `cloud-native-notes:3.0`, which was never built	`minikube image ls | grep cloud-native-notes`	Build a new tag (`v8`) and point the Deployment and manifest at it

#	Symptom	Cause	How it was found	Fix
4	`502 Bad Gateway`; logs repeat *AUTH called without any password configured*	The backend sent the Secret's password to an old Redis that had none	`kubectl logs` on each backend pod	Run week 6's Redis, which starts with `--requirepass $(REDIS_PASSWORD)`
5	Week 6's Redis files had never been applied	Week 6 keeps manifests in subfolders, and `kubectl apply -f k8s/` skips subfolders silently	`ls k8s/` showed `backend/ frontend/ redis/`	`kubectl apply -R -f k8s/`

Lessons for participants: check endpoints and `env` before blaming the network, read `kubectl logs` on every pod, use a new image tag for every code change, and always use `-R` when manifests live in subfolders.

Why is the LoadBalancer pending?

A LoadBalancer Service shows `EXTERNAL-IP <pending>` because nothing on a laptop can create a load balancer. Kubernetes only *requests* one; a cloud provider is supposed to fulfil the request.

On AWS, GCP or Azure, a component called the *cloud controller manager* sees the request, creates a real load balancer, and writes its IP back into the Service. A local cluster has no such component, so the request waits forever.

See it for yourself:

```
kubectl create deployment hello --image=nginx -n cloud-native-notes
kubectl expose deployment hello --type=LoadBalancer --port=80 -n cloud-
native-notes
kubectl get svc hello -n cloud-native-notes        # EXTERNAL-IP <pending>
```

Now start `minikube tunnel` in another terminal and run the last command again. The external IP becomes `127.0.0.1`: the tunnel plays the role of the cloud provider. Stop the tunnel and it goes back to pending.

This is also why the README's cloud manifest for ingress-nginx leaves its controller Service pending on Kind: that manifest asks for a LoadBalancer. Other local fixes are MetalLB and `cloud-provider-kind`.

Clean up afterwards:

```
kubectl delete deployment,svc hello -n cloud-native-notes
```

Troubleshooting

Most problems in this lab come from four places: the image, the Service selector, the Host header, or the Mac network. Start with `kubectl describe` and `kubectl logs` on whatever looks wrong.

Symptom	Likely cause	Fix
`minikube start` fails with a Docker error	Docker Desktop is not running	Open Docker Desktop, wait for "running", retry
`ImagePullBackOff` / `ErrImagePull` / `ErrImageNeverPull`	Image not built inside Minikube, wrong tag, or tag is `:latest`	`minikube image build -t <name:tag> ./backend` with the exact tag from the manifest; set `imagePullPolicy: IfNotPresent`
`CrashLoopBackOff`	The app fails on start, for example `ModuleNotFoundError: No module named 'app.metrics'` because a file is missing from `backend/app`	Read `kubectl logs deploy/backend -n cloud-native-notes --previous`, fix the code, rebuild (see below)
`CreateContainerConfigError`	A ConfigMap or Secret named in `envFrom` is missing	`kubectl get configmap,secret -n cloud-native-notes`; apply `k8s/` again
`curl $(minikube ip):<port>` hangs	Docker driver on macOS hides the node IP	Use Method A, B or C from Lab 4
`Connection refused` on `localhost:<NODEPORT>`	Command ran on the Mac, not the node	`minikube ssh` first (prompt `docker@minikube`)
`400` — *plain HTTP request was sent to HTTPS port*	HTTP sent to the 443 NodePort or the second `--url` URL	Use the HTTP port / first URL

Symptom	Likely cause	Fix
`404 Not Found` from nginx	Missing or wrong Host header, wrong path, or wrong `ingressClassName`	Add `-H "Host: notes.local"`; check `kubectl describe ingress`
`301 Moved Permanently` on `/api`	Frontend NGINX uses `location /api/`	Request `/api/` or `/api/notes`
`503 Service Temporarily Unavailable`	Service has no ready pods	`kubectl get endpoints -n cloud-native-notes`; match Service selector to pod labels
`failed calling webhook "validate.nginx.ingress.kubernetes.io"`	Ingress applied while the controller was still starting	Run the `kubectl wait` from Lab 3, then `kubectl apply -f k8s/` again
Ingress `ADDRESS` stays empty	No controller, or class name mismatch	`kubectl get pods -n ingress-nginx`; `kubectl get ingressclass`
`notes.local` does not open in the browser	`/etc/hosts` line missing, tunnel stopped, or wrong (HTTPS) port	`grep notes.local /etc/hosts`; keep the tunnel running; type the full `http://` address
`grep: /etc/nginx/nginx.conf: No such file or directory`	Ran only the `grep`, on the Mac	Run the whole `kubectl exec ... -- grep ...` line
`exec: "curl": executable file not found`	Frontend image has no curl	Use the `curlimages/curl` pod from Lab 5
`verify-week6.sh` fails checks 4, 8–11	Script expects the Kind setup	Follow *Week 6 — Pass the verification script on Minikube*

After fixing code, rebuild the image. The cluster keeps running the old image until you do. Run every command from the week folder (`week5` or `week6`): a build run from the wrong folder prints nothing and builds nothing. Use a new tag, then switch the Deployment to it:

```
minikube image build -t cloud-native-notes:v6 ./backend
minikube ssh -- docker run --rm cloud-native-notes:v6 ls /app/app   # confirm the file is inside
kubectl set image deployment/backend -n cloud-native-notes '*=cloud-native-notes:v6'
```

Also update the `image:` line in `k8s/backend-deployment.yaml`, or the next `kubectl apply -f k8s/` brings the old tag back.

When all else fails: `minikube delete && minikube start --driver=docker` gives a clean cluster in about two minutes.

Extra challenges and cleanup

These are take-home tasks from the README, with hints adapted to Minikube.

1. Gateway API. Read why it is replacing Ingress: roles are split (platform team owns the `Gateway`, app teams own `HTTPRoute`s), and traffic splitting and header matching are built in instead of vendor annotations. To try it:

- Install the Gateway API CRDs and one implementation, for example NGINX Gateway Fabric or Envoy Gateway, following that project's install page.
- Write a `Gateway` listening on port 80 and an `HTTPRoute` that sends host `notes.local`, path `/api` to Service `backend` port 3000.
- Expose it with `minikube tunnel` exactly as in Lab 4, Method C.

2. TLS. Let's Encrypt only issues certificates for a public domain it can reach, so it will not work for `notes.local`. Locally, practise the same flow with cert-manager and a self-signed issuer, or with `mkcert`, then add a `tls:` block to the Ingress. Use Let's Encrypt later on a cloud cluster with a real domain.

3. Browser access via `notes.local`. On Minikube this is Lab 4, Method C. On Kind it needs the cluster recreated with `extraPortMappings` for ports 80 and 443.

Cleanup when you are done:

```
kubectl delete -f k8s/          # remove the app
minikube stop                   # pause the cluster, keep its state
minikube delete                 # remove the cluster completely
sudo sed -i '' '/notes.local/d' /etc/hosts   # remove the hosts entry
```

Facilitator notes and cheat sheet

Run through every lab yourself on a clean Mac the day before, using the real image name from `k8s/`.

- Ask participants to finish the installation section before the session and send a screenshot of `minikube status`.
- Keep two spare terminals open during Lab 4: `minikube service --url` and `minikube tunnel` both block their terminal.
- Teach everyone to read their prompt before each command: `yomex@... week5 %` is the Mac, `docker@minikube:~$` is the node. Most errors today came from running the right command in the wrong place or folder.

- Don't put NodePort numbers on slides: each person's are random. Have them look up their own.
- At each checkpoint, ask everyone to post their output in the chat before moving on.
- Draw the traffic diagram on a whiteboard and add each piece as the labs create it.
- Live demo: `kubectl get endpoints backend -n cloud-native-notes -w` in one terminal, then scale the backend to 3 in another and watch a third IP appear.
- Good wrap-up questions: What breaks if the controller pod dies? What does the Host header do? Why is the backend not reachable from the Mac? Why does `/api` give 301?

Command cheat sheet

Task	Command
Start cluster	`minikube start --driver=docker`
Build image into cluster	`minikube image build -t <name:tag> ./backend` (from the week folder)
Deploy app	`kubectl apply -f k8s/`
Enable Ingress	`minikube addons enable ingress`
Everything in the app namespace	`kubectl get all -n cloud-native-notes`
Pods behind each Service	`kubectl get endpoints -n cloud-native-notes`
Ingress details	`kubectl describe ingress -n cloud-native-notes`
Your HTTP NodePort	`kubectl get svc ingress-nginx-controller -n ingress-nginx -o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}'`
Controller logs	`kubectl logs -n ingress-nginx deploy/ingress-nginx-controller`
Crash reason	`kubectl logs deploy/backend -n cloud-native-notes --previous`
Controller URL on Mac	`minikube service ingress-nginx-controller -n ingress-nginx --url` (first URL = HTTP)
Browser access	`minikube tunnel` + `127.0.0.1 notes.local` in `/etc/hosts`
Shell on the node	`minikube ssh`
Week 6 checks	`./verify-week6.sh`

Task	Command
Web dashboard	`minikube dashboard`
Reset everything	`minikube delete`